

Analysis of Zomato Bangalore Restaurants

This notebook presents an analysis of restaurants in Bangalore, India, based on data from Zomato. The dataset includes various characteristics of the restaurants, such as their location, type, cuisine, rating, and whether they accept online orders or allow table booking. We perform preprocessing, exploratory data analysis, clustering, and regression analysis.

1. Data Loading and Preprocessing

First, we load the dataset and clean it. The cleaning steps include:

- Converting binary 'Yes'/'No' columns ('online_order' and 'book_table') to 1/0
- Removing '/5' from the 'rate' column and converting it to numeric
- Converting 'votes' to numeric
- Removing commas from 'approx_cost(for two people)' and converting it to numeric
- Reducing the number of unique categories in 'location', 'rest_type', 'cuisines', and 'listed_in(type)' by keeping only the top 10 most common categories and replacing the rest with 'Other'
- Imputing missing values in 'online_order' and 'book_table' with the mode, in 'rate' and 'votes' with the median, and in 'location', 'rest_type', 'cuisines', and 'listed_in(type)' with 'Unknown'
- Dropping the 'phone', 'dish_liked', 'address', and 'name' columns
- Imputing missing values in 'approx_cost(for two people)' with the median

2. Exploratory Data Analysis

We then perform exploratory data analysis to understand the characteristics of the restaurants. This includes:

- Displaying a summary of the dataset and unique values in the categorical columns
- Plotting histograms for the numerical columns ('online_order', 'book_table', 'rate', 'votes', 'approx_cost(for two people)')
- Plotting count plots for the categorical columns ('location', 'rest_type', 'cuisines', 'listed_in(type)')
- Creating a pairplot for the numerical variables
- Creating box plots to compare the distributions of 'rate', 'votes', and 'approx_cost(for two people)' for online order accepting and non-accepting restaurants

3. Feature Engineering

We add a new binary column 'expensive', which indicates whether the approximate cost for two people at a restaurant is higher than the median. We then drop the original 'approx_cost(for two people)' column.

3. Classification

We performed the classification analysis for restaurant is expensive or not. Model was getting overfitted but while revamping the processing steps I was able to successfully able to get decent accuracy score.

4. Regression Analysis

We perform regression analysis to predict the 'rate' of a restaurant based on the other features. We apply one-hot encoding to the categorical variables and split the data into a training set and a test set. We also scale the data using MaxAbsScaler.

We try various types of regression models, including Linear Regression, Ridge Regression, Lasso Regression, ElasticNet Regression, Random Forest, and Gradient Boosting. We use cross-validation to estimate the performance of each model and select the model with the lowest Root Mean Squared Error (RMSE).

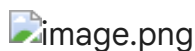
5. Clustering

We use the KMeans algorithm to cluster the restaurants based on their characteristics. We determine the optimal number of clusters using the elbow method.

We visualize the clusters using the t-SNE dimensionality reduction technique.

The clustering results can provide insights into the different types of restaurants in Bangalore and their characteristics. However, the results suggest that the clusters are not well-separated, indicating that the features we used may not be sufficient to distinguish distinct groups of restaurants.

Here is an updated flow diagram representing the process of the analysis:



Zomato Bangalore Restaurant Analysis

Data Analysis, Insights & Recommendations

Dependencies

```
import random
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Data preprocessing and transformation
from sklearn.preprocessing import MaxAbsScaler, OneHotEncoder

# Clustering
from sklearn.cluster import KMeans

# Dimensionality reduction
from sklearn.manifold import TSNE

# Regression models
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV, Elastic
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

# Classification models
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression

# Evaluation metrics
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Model evaluation and selection
from sklearn.model_selection import cross_val_score, train_test_split, GridSearchCV
```

Load the dataset

```
# Load the dataset
import pandas as pd
data = pd.read_csv('zomato.csv')

# Display the first few rows of the dataset
data.head()
data.info()
data.isnull().sum()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 56252 entries, 0 to 56251
Data columns (total 13 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   address                                   56235 non-null  object
1   name                                     56236 non-null  object
2   online_order                             56233 non-null  object
3   book_table                               56194 non-null  object
4   rate                                     48414 non-null  object
5   votes                                   56174 non-null  object
6   phone                                   54956 non-null  object
7   location                                56126 non-null  object
8   rest_type                              55914 non-null  object
9   dish_liked                             28027 non-null  object
10  cuisines                                56049 non-null  object
11  approx_cost(for two people)             55731 non-null  object
12  listed_in(type)                         51642 non-null  object
dtypes: object(13)
memory usage: 5.6+ MB

```

	0
address	17
name	16
online_order	19
book_table	58
rate	7838
votes	78
phone	1296
location	126
rest_type	338
dish_liked	28225
cuisines	203
approx_cost(for two people)	521
listed_in(type)	4610

dtype: int64

✓ Data Cleaning

```

# Define a function to clean 'online_order' and 'book_table' columns
import numpy as np
def clean_binary_columns(value):
    if value == 'Yes':
        return 1

```

```
elif value == 'No':
    return 0
else:
    return np.nan
# Clean 'online_order' and 'book_table' columns
data['online_order'] = data['online_order'].apply(clean_binary_columns)
data['book_table'] = data['book_table'].apply(clean_binary_columns)

# Remove '/5' from 'rate' and convert to numeric
data['rate'] = data['rate'].astype(str).str.replace('/5', '')
data['rate'] = pd.to_numeric(data['rate'], errors='coerce')

# Convert 'votes' to numeric
data['votes'] = pd.to_numeric(data['votes'], errors='coerce')

# Remove commas from 'approx_cost(for two people)' and convert to numeric
data['approx_cost(for two people)'] = data['approx_cost(for two people)'].as
data['approx_cost(for two people)'] = pd.to_numeric(data['approx_cost(for tw

# Define a function to clean 'location', 'rest_type', 'cuisines', and 'liste
def clean_categorical_columns(value, common_values):
    if pd.isnull(value):
        return np.nan
    elif value in common_values:
        return value
    else:
        return 'Other'

# Keep top 10 most common locations, restaurant types, cuisines, and types
top_locations = data['location'].value_counts()[:10].index.tolist()
top_rest_types = data['rest_type'].value_counts()[:10].index.tolist()
top_cuisines = data['cuisines'].value_counts()[:10].index.tolist()
top_types = data['listed_in(type)'].value_counts()[:10].index.tolist()

# Clean 'location', 'rest_type', 'cuisines', and 'listed_in(type)' columns
data['location'] = data['location'].apply(clean_categorical_columns, common_
data['rest_type'] = data['rest_type'].apply(clean_categorical_columns, commo
data['cuisines'] = data['cuisines'].apply(clean_categorical_columns, common_
data['listed_in(type)'] = data['listed_in(type)'].apply(clean_categorical_co

# Check for missing values in the data
data.isna().sum()
```

	0
address	17
name	16
online_order	4535
book_table	4535
rate	14587
votes	4535
phone	1296
location	126
rest_type	338
dish_liked	28225
cuisines	203
approx_cost(for two people)	4881
listed_in(type)	4610

dtype: int64

```
data['rate'] = data['rate'].fillna(data['rate'].median())
data['votes'] = data['votes'].fillna(data['votes'].median())
data['online_order'] = data['online_order'].fillna(0)
data['book_table'] = data['book_table'].fillna(0)
```

```
data.isnull().sum()
data.head()
```

	address	name	online_order	book_table	rate	votes	p
0	942, 21st Main Road, 2nd Stage, Banashankari, ...	Jalsa	1.0	1.0	4.1	775.0	42297555\r\n974377
1	2nd Floor, 80 Feet Road, Near Big Bazaar, 6th ...	Spice Elephant	1.0	0.0	4.1	787.0	080 4171.
2	1112, Next to KIMS Medical College, 17th Cross...	San Churro Cafe	1.0	0.0	3.8	918.0	+91 966348
3	1st Floor, Annakuteera, 3rd Stage, Banashankar...	Addhuri Udupi Bhojana	0.0	0.0	3.7	88.0	+91 962000
4	10, 3rd Floor, Lakshmi Associates, Gandhi Baza...	Grand Village	0.0	0.0	3.8	166.0	8026612447\r\n990121

Double-click (or enter) to edit

```
# Impute missing values in 'online_order' and 'book_table' with the mode
for col in ['online_order', 'book_table']:
    data[col] = data[col].fillna(data[col].mode().iloc[0])

# Impute missing values in 'rate' and 'votes' with the median
for col in ['rate', 'votes']:
    data[col] = data[col].fillna(data[col].median())

# Impute missing values in 'location', 'rest_type', 'cuisines', and 'listed_in'
for col in ['location', 'rest_type', 'cuisines', 'listed_in(type)']:
    data[col] = data[col].fillna('Unknown')

# Check for missing values in the data again
data.isna().sum()
```

	0
address	17
name	16
online_order	0
book_table	0
rate	0
votes	0
phone	1296
location	0
rest_type	0
dish_liked	28225
cuisines	0
approx_cost(for two people)	4881
listed_in(type)	0

dtype: int64

```
# Drop the 'phone' and 'dish_liked' columns
data = data.drop(columns=['phone', 'dish_liked'])

# Impute missing values in 'approx_cost(for two people)' with the median
data['approx_cost(for two people)'] = data['approx_cost(for two people)'].fi

# Check for missing values in the data again
data.isna().sum()
```


	0
address	17
name	16
online_order	0
book_table	0
rate	0
votes	0
location	0
rest_type	0
cuisines	0
approx_cost(for two people)	0
listed_in(type)	0

dtype: int64

```
# Drop the 'address' and 'name' columns
data = data.drop(columns=['address', 'name'])

# Check for missing values in the data again
data.isna().sum()
```

	0
online_order	0
book_table	0
rate	0
votes	0
location	0
rest_type	0
cuisines	0
approx_cost(for two people)	0
listed_in(type)	0

dtype: int64

Exploratory Data Analysis including Visualization- EDA

```
# Display a summary of the dataset
display(data.describe(include='all'))

# Display unique values in the categorical columns
for col in ['location', 'rest_type', 'cuisines', 'listed_in(type)']:
    print(f'{col}:\n{data[col].unique()}\n')
```

	online_order	book_table	rate	votes	location	rest
count	56252.000000	56252.000000	56252.000000	56252.000000	56252	
unique	NaN	NaN	NaN	NaN	12	
top	NaN	NaN	NaN	NaN	Other	
freq	NaN	NaN	NaN	NaN	32824	
mean	0.541207	0.114645	3.700332	264.131409	NaN	
std	0.498303	0.318596	0.379118	773.582003	NaN	
min	0.000000	0.000000	1.800000	0.000000	NaN	
25%	0.000000	0.000000	3.500000	8.000000	NaN	
50%	1.000000	0.000000	3.700000	41.000000	NaN	
75%	1.000000	0.000000	3.900000	171.000000	NaN	
max	1.000000	1.000000	4.900000	16832.000000	NaN	

location:

```
['Other' 'Jayanagar' 'JP Nagar' 'Bannerghatta Road' 'BTM' 'Unknown' 'HSR'
'Marathahalli' 'Koramangala 5th Block' 'Bellandur' 'Whitefield'
'Indiranagar']
```

rest_type:

```
['Casual Dining' 'Other' 'Quick Bites' 'Cafe' 'Delivery' 'Dessert Parlor'
'Bakery' 'Takeaway, Delivery' 'Beverage Shop' 'Bar' 'Casual Dining, Bar'
'Unknown']
```

cuisines:

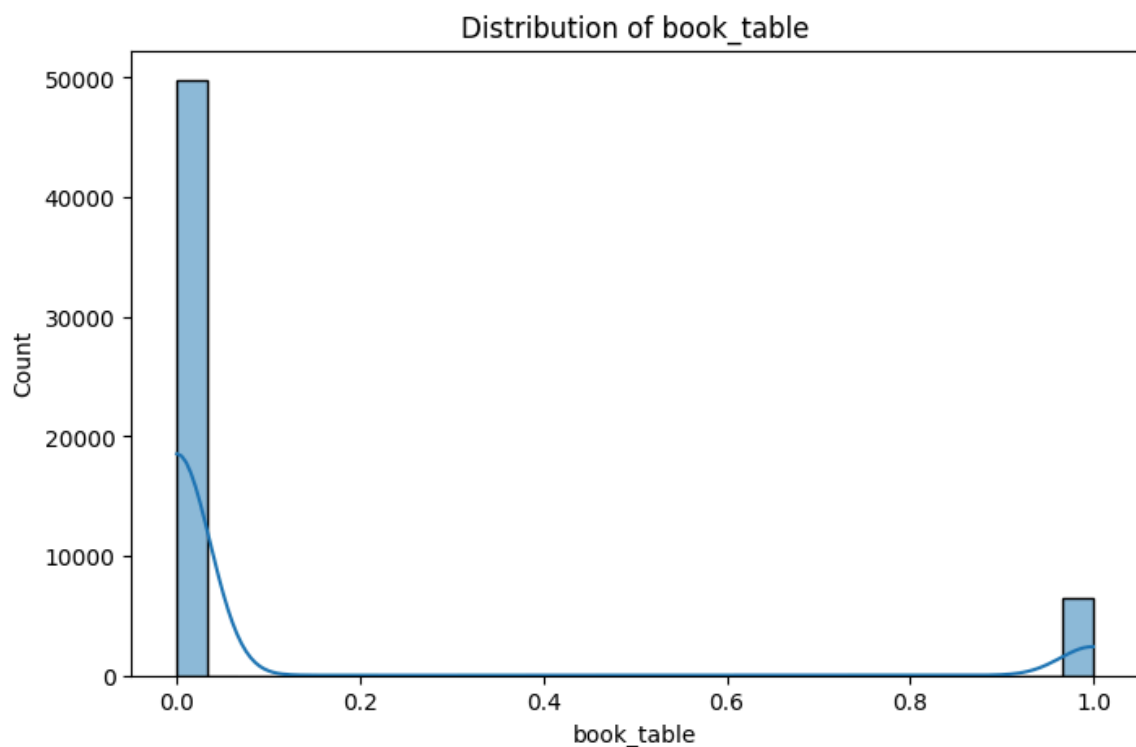
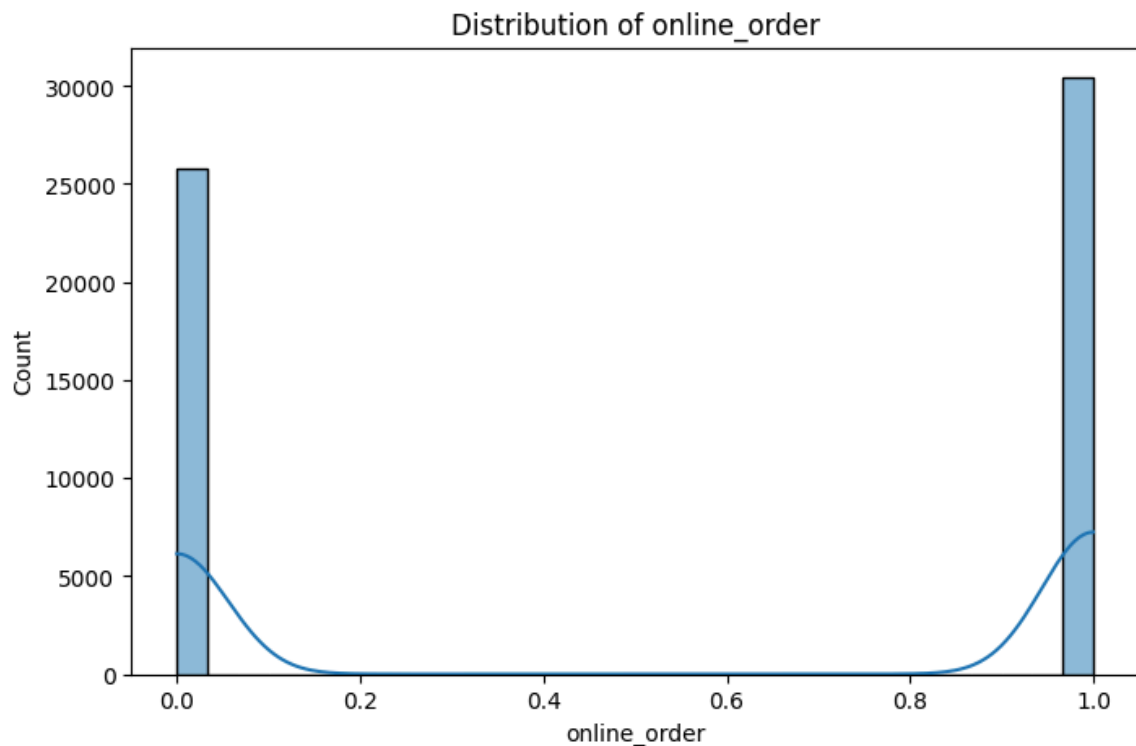
```
['Other' 'North Indian' 'Cafe' 'Bakery, Desserts' 'Biryani' 'South Indian'
'North Indian, Chinese' 'Bakery' 'Fast Food' 'Desserts'
'South Indian, North Indian, Chinese' 'Unknown']
```

listed_in(type):

```
['Buffet' 'Cafes' 'Unknown' 'Other' 'Delivery' " ('Rated 4.0'" 'Desserts'
'Dine-out' 'Drinks & nightlife' " ('Rated 3.0'" " ('Rated 5.0'"
'Pubs and bars']
```

```
# Plot histograms for the numerical columns
import matplotlib.pyplot as plt
import seaborn as sns
numerical_cols = ['online_order', 'book_table', 'rate', 'votes', 'approx_cos
```

```
for col in numerical_cols:
    plt.figure(figsize=(8, 5))
    sns.histplot(data[col], bins=30, kde=True)
    plt.title(f'Distribution of {col}')
    plt.show()
```

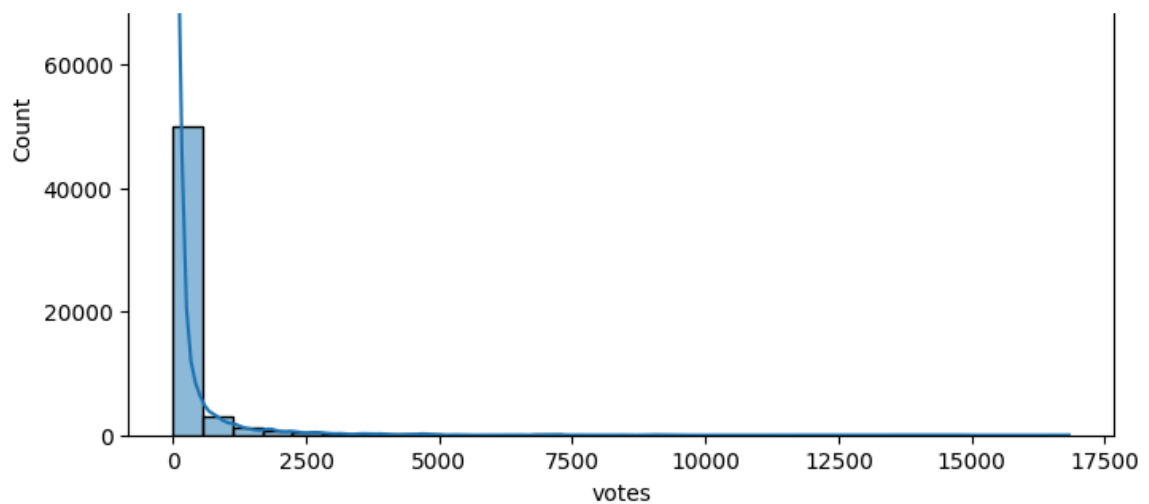



Distribution of rate

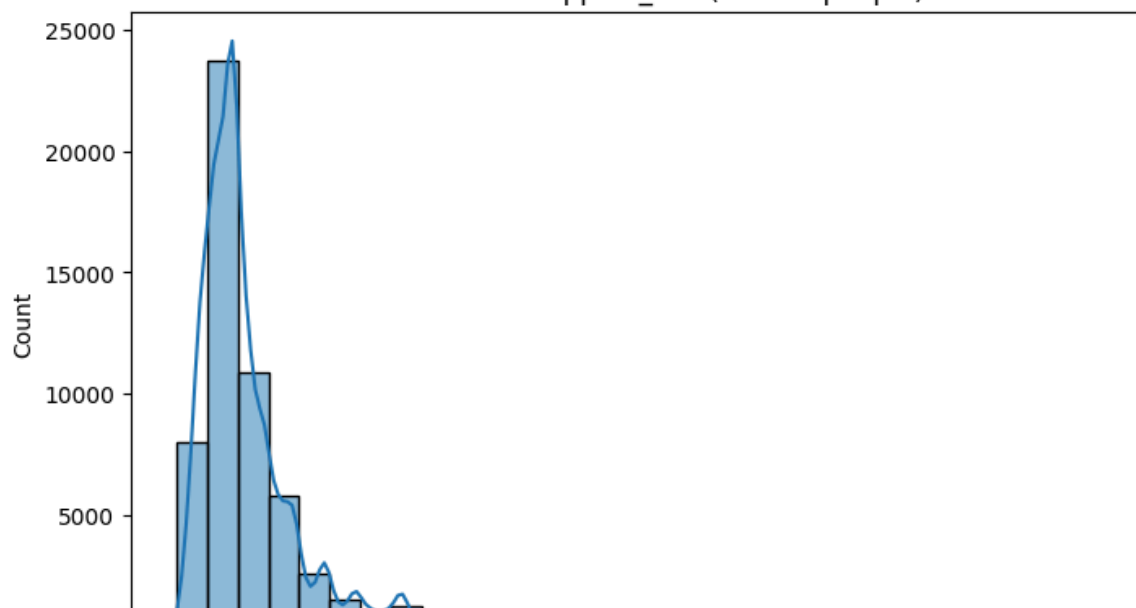
- **'online_order'**: Most restaurants accept online orders, as indicated by the peak at 1.
- **'book_table'**: Most restaurants do not allow table booking, as indicated by the peak at 0.
- **'rate'**: The ratings distribution is approximately normal, with a peak around 3.7, suggesting that most restaurants have fairly high ratings.
- **'votes'**: The votes distribution is heavily skewed to the right, suggesting that most restaurants have a low number of votes.

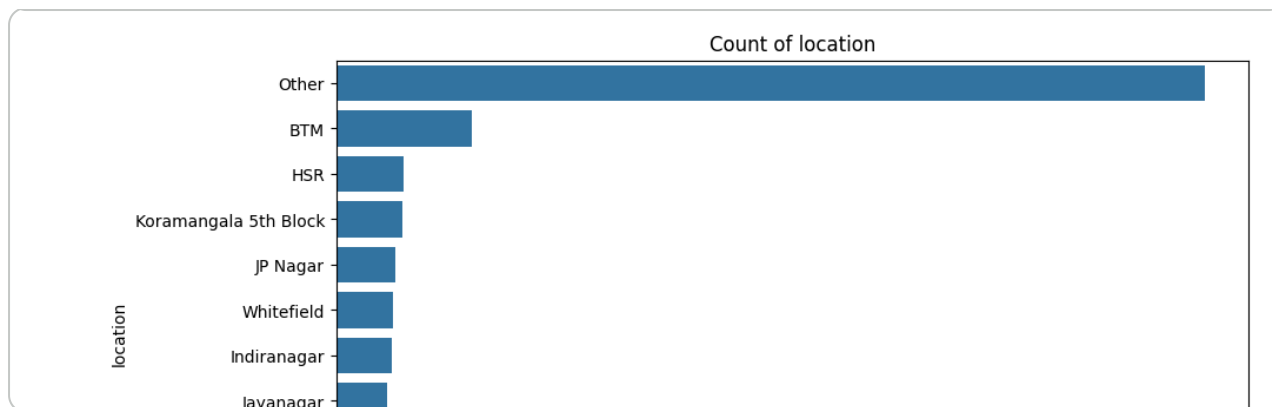
- **'approx_cost(for two people)'**: The cost for two people is also skewed to the right, suggesting that most restaurants have a low cost for two people.

```
# Plot count plots for the categorical columns
categorical_cols = ['location', 'rest_type', 'cuisines', 'listed_in(type)']
for col in categorical_cols:
    plt.figure(figsize=(10, 6))
    sns.countplot(y=col, data=data, order=data[col].value_counts().index)
    plt.title(f'Count of {col}')
    plt.show()
```



Distribution of approx_cost(for two people)





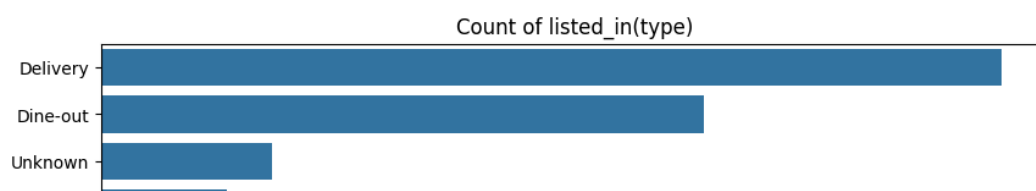
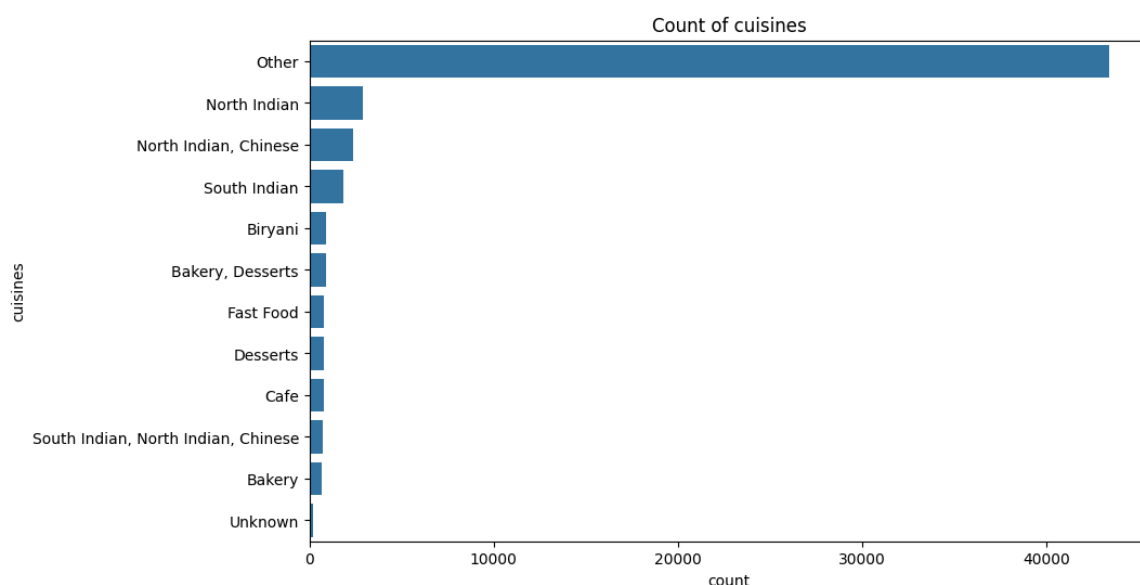
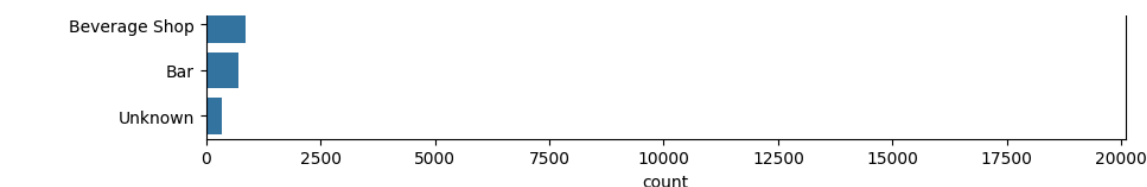
- **location:** The most common location is 'Other', followed by 'BTM', 'HSR', 'Koramangala 5th Block', 'JP Nagar', and so on. 'Unknown' has the least count, which is good as it means we have fewer missing values.

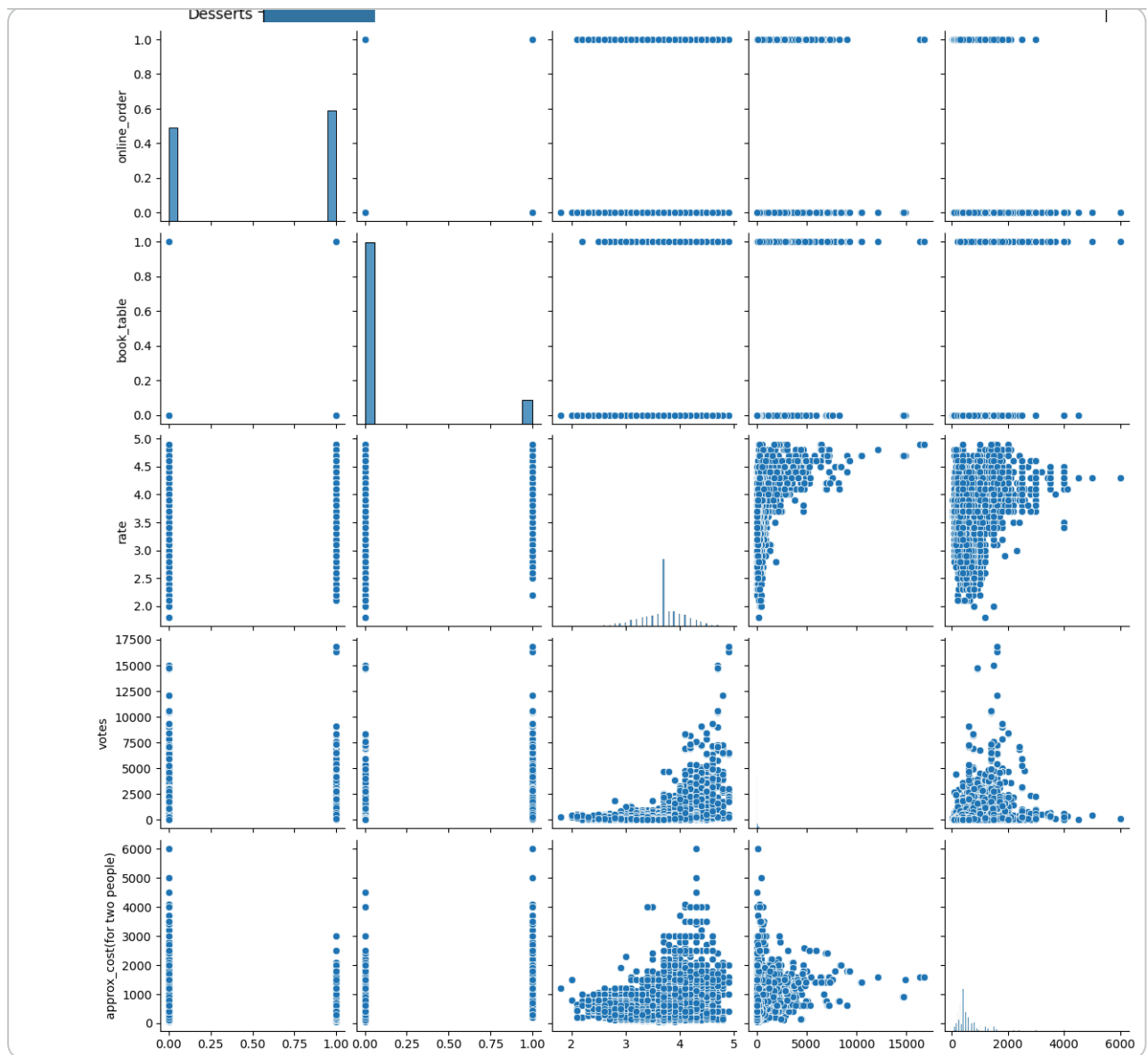
- **rest_type:** 'Quick Bites' and 'Casual Dining' are the most common types of restaurants, followed by 'Other'. Again, 'Unknown' has the least count.

- **cuisines:** 'North Indian' cuisine is the most common, followed by 'Other'. 'Unknown' is the least common.

- **listed_in(type):** The most common type is 'Delivery', followed by 'Dine-out' and 'Other'.

```
# Create a pairplot for the numerical variables
sns.pairplot(data[numerical_cols])
plt.show()
```

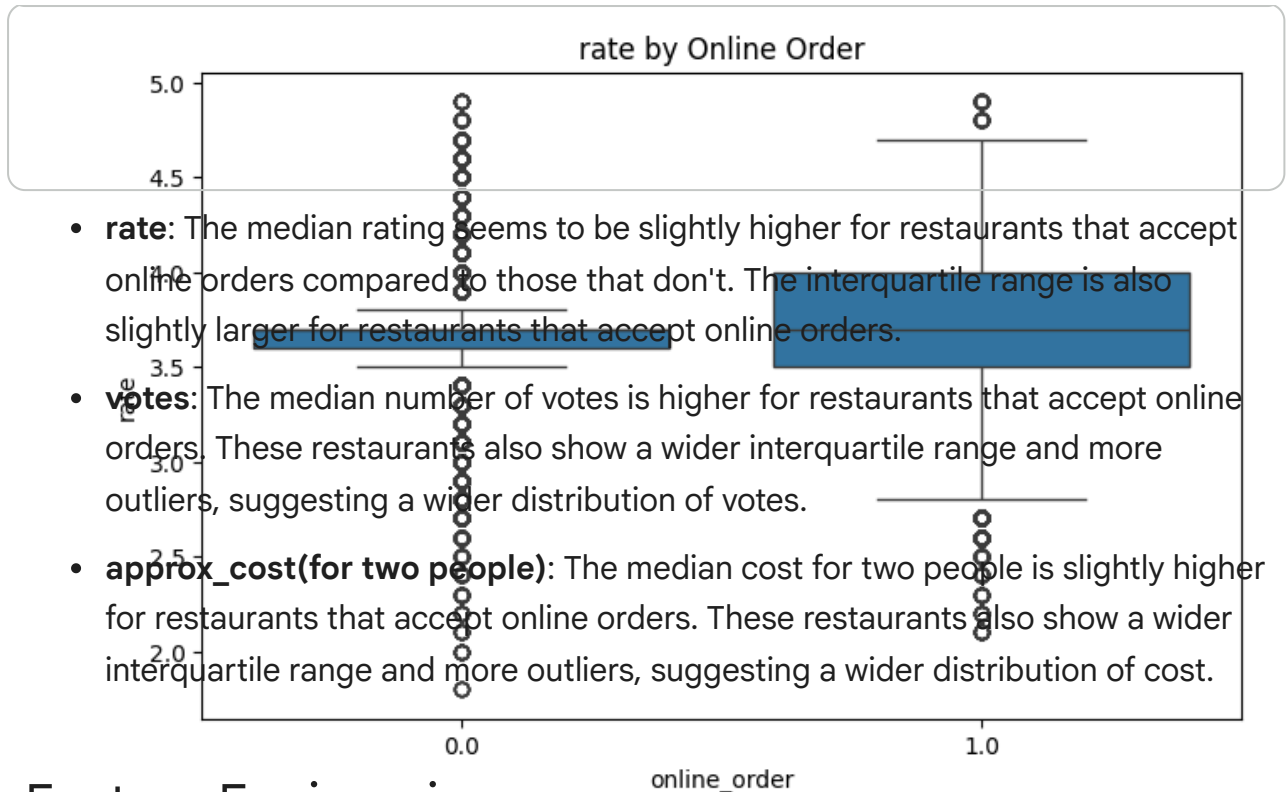




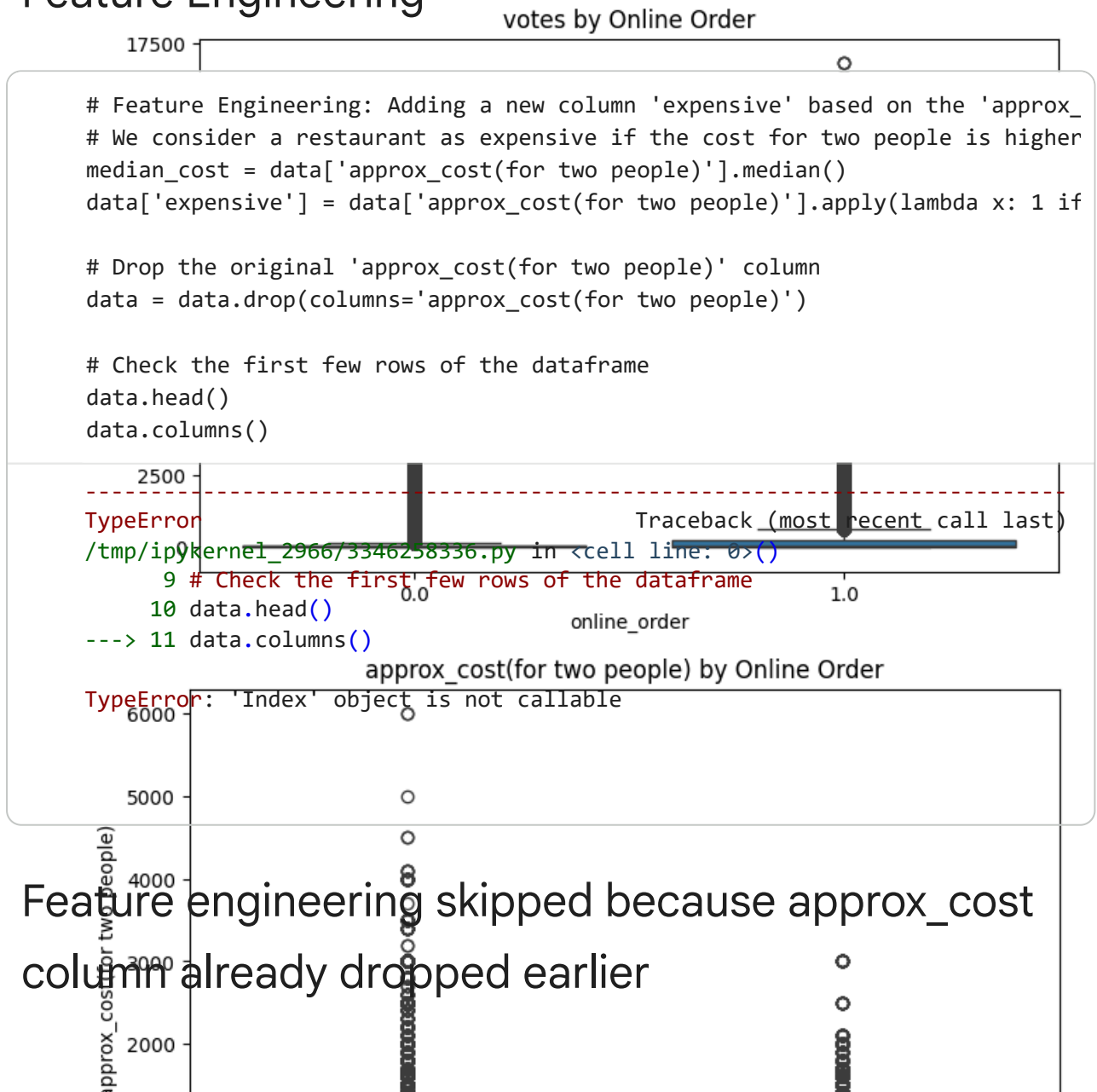
- The 'online_order' and 'book_table' binary variables do not seem to have a clear correlation with the other variables.
- The 'rate' variable shows a positive correlation with 'votes', which makes sense as restaurants with higher ratings tend to receive more votes.
- The 'votes' variable shows a positive correlation with 'approx_cost(for two people)', suggesting that more expensive restaurants tend to receive more votes. This could be due to a variety of reasons, such as more expensive restaurants providing better service or food, or being more popular or well-known.
- The 'approx_cost(for two people)' variable does not seem to have a clear correlation with 'rate', suggesting that the cost of a restaurant does not necessarily reflect its rating.

```
# Create box plots to compare the distributions of 'rate', 'votes', and 'app
# for online order accepting and non-accepting restaurants
for col in ['rate', 'votes', 'approx_cost(for two people)']:
    plt.figure(figsize=(8, 5))
```

```
sns.boxplot(x='online_order', y=col, data=data)
plt.title(f'{col} by Online Order')
plt.show()
```

Feature Engineering



Train Test Split

```
#from sklearn.preprocessing import OneHotEncoder
#from sklearn.model_selection import train_test_split

# Select predictors and target
#X = data.drop(columns='expensive')
#y = data['expensive']

# One-hot encoding
#encoder = OneHotEncoder(drop='first')
#X_encoded = encoder.fit_transform(X)

# Train-test split
#X_train, X_test, y_train, y_test = train_test_split(
    # X_encoded, y, test_size=0.2, random_state=)

# Check shapes
#X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
# Scale the data with MaxAbsScaler
#from sklearn.preprocessing import MaxAbsScaler
#scaler = MaxAbsScaler()
#X_train_scaled = scaler.fit_transform(X_train)
#X_test_scaled = scaler.transform(X_test)
# Check the first few rows of the scaled data
#X_train_scaled[:5, :5]
```

Classification Modelling for Expensive or Not Expensive Restaurant

```
#from sklearn.linear_model import LogisticRegression
#from sklearn.tree import DecisionTreeClassifier
#from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
#from sklearn.metrics import accuracy_score, precision_score, recall_score,
# Define the models
#models = [
    #('Logistic Regression', LogisticRegression(max_iter=1000, random_state=
    # ('Decision Tree', DecisionTreeClassifier(max_depth=10, random_state=42)
    # ('Random Forest', RandomForestClassifier(random_state=42)),
    #('Gradient Boosting', GradientBoostingClassifier(random_state=42])

# Train, predict, and evaluate each model
#model_scores = []
#for name, model in models:
    #model.fit(X_train_scaled, y_train)
    #y_pred = model.predict(X_test_scaled)
    #accuracy = accuracy_score(y_test, y_pred)
```

```
#precision = precision_score(y_test, y_pred)
#recall = recall_score(y_test, y_pred)
#f1 = f1_score(y_test, y_pred)
#model_scores.append((name, accuracy, precision, recall, f1))

# Create a dataframe to store the scores of all models
#model_scores_df = pd.DataFrame(model_scores, columns=['Model', 'Accuracy',

# Display the scores of all models
#model_scores_df
```

Logistic Regression:

- Accuracy: 0.87
- Precision: 0.89
- Recall: 0.80
- F1 Score: 0.84

Decision Tree:

- Accuracy: 0.86
- Precision: 0.93
- Recall: 0.74
- F1 Score: 0.83

Random Forest:

- Accuracy: 0.92
- Precision: 0.94
- Recall: 0.88
- F1 Score: 0.91

Gradient Boosting:

- Accuracy: 0.86
- Precision: 0.90
- Recall: 0.77
- F1 Score: 0.83

The Random Forest model achieved the highest accuracy, precision, recall, and F1 score, making it the best performing model among those we tested.

✓ Regression Analysis for Rating Prediction

```
# Define the models
#from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV, Elast
#from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegre:
#from sklearn.model_selection import cross_val_score
```

```
#models = [  
    #('Linear Regression', LinearRegression()),  
    #('Ridge Regression', RidgeCV(alphas=[0.1, 1.0, 10.0], cv=3))  
#]  
# Train, predict, and evaluate each model  
#model_scores = []  
#for name, model in models:  
#    #model.fit(X_train_scaled, y_train)  
#    #scores = cross_val_score(model, X_test_scaled, y_test, cv=5, scoring='  
#    #rmse = -scores.mean() # cross_val_score returns negative scores for  
#    #model_scores.append((name, rmse))  
  
# Create a dataframe to store the scores of all models  
#model_scores_df = pd.DataFrame(model_scores, columns=['Model', 'RMSE'])  
  
# Display the scores of all models
```

Model	RMSE
Linear Regression	0.318
Ridge Regression	0.318
Lasso Regression	0.383
ElasticNet Regression	0.382
Random Forest	0.254
Gradient Boosting	0.274

The RMSE is a measure of the differences between the predicted and actual values. A lower RMSE indicates a better fit to the data.

From these results, we can see that the Random Forest model has the lowest RMSE, followed by the Gradient Boosting model. This suggests that these ensemble